Bitácora de uso de inteligencia artificial

> Completa el nombre del integrante y el hash del commit cuando el equipo integre estos cambios. El repositorio conserva el commit inicial de plantilla; esta implementación todavía no está comprometida.

Entrada #1

- Fecha: 2026-10-28
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Iniciar el desarrollo de la aplicación móvil para conductores a partir de la especificación funcional y técnica.
- Prompt: "sigue las intrucciones del md para el desarrollo de la aplicaion"
- Resumen de la respuesta: Se creó el cliente Expo Router con cálculo de tarifas, registro y flujo de viajes demostrativo.
- Qué se usó / modificó / descartó: Se usó el ejemplo de tarifa de 8,5 km/22 minutos. El backend de demostración inicial fue reemplazado posteriormente por la API Mongo descrita en la Entrada #2.
- Archivos y commit: frontend/, backend/, README.md — cambios sin commit; completar el hash al integrarlos.
- Verificación: Pruebas iniciales de tarifas, validación TypeScript, lint y compilación Android.
- Aprendizaje: Las claves secretas deben permanecer en el backend; no guardar URI de Mongo, contraseñas ni tokens dentro del bundle móvil.

Entrada #2

- Fecha: 2026-10-28
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Separar frontend y backend y persistir perfiles de conductor usando MongoDB.
- Prompt: "okey para la persistencia de datos vamos a utilizar mongodb [URI omitida intencionalmente por seguridad] con la clave de conecion a cluster y necesito separar el backend del fronted en folders separados para tener mejor orden"
- Resumen de la respuesta: Se creó una API Express con MongoDB Atlas, autenticación con contraseña hasheada y token de sesión, validación de perfiles y almacenamiento privado de imágenes en GridFS. El cliente ahora registra conductores, inicia/cierra sesión y carga/edita el perfil.
- Qué se usó / modificó / descartó: Se guardan los documentos relacionados al perfil en el mismo documento MongoDB y las imágenes fuera del documento en GridFS; se exige aprobación administrativa y se evita guardar secretos en el repositorio. La URI enviada no se guardó ni se usó como credencial.
- Archivos y commit: frontend/, backend/, README.md, .gitignore — cambios sin commit; completar el hash al integrarlos.
- Verificación: Backend npm test (4/4) y build TypeScript; frontend npm test (3/3), npx tsc --noEmit, npx expo lint, npx expo-doctor (20/20) y bundle Android.
- Aprendizaje: La URI de Atlas debe respetar el formato mongodb+srv://usuario:contraseña@host/base y la contraseña debe codificarse para URL si contiene caracteres reservados. La siguiente verificación requiere conectar un clúster de pruebas sin exponer su credencial.

Entrada #3

- Fecha: 2026-10-31
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Leer la especificación PDF del conductor y alinear las funcionalidades, integraciones y flujos existentes.
- Prompt: "lee todo el pdf con las especificasiones de codigo, funcionalidades, flujos y herramientas solicitadas y propuestas, y modifica lo que tenemos para que sigua lo que nos piden"

- Resumen de la respuesta: Se extrajo y leyó el PDF completo. Se implementaron ubicación persistida, Socket.IO con MongoDB Change Streams, FCM, Google Maps/Distance Matrix, precio compartido, checkout Mercado Pago, rechazo por vencimiento, filtros de historial y secciones de perfil/configuración.
- Qué se usó / modificó / descartó: Se usó MongoDB Change Streams, FCM, Google Maps y Mercado Pago de prueba como recomienda la especificación; OpenRouteService queda como alternativa opcional. Stripe y el cliente de pasajero no se añadieron porque Mercado Pago es la pasarela recomendada y el alcance del repositorio es el conductor.
- Archivos y commit: frontend/, backend/, shared/pricing.ts, README.md, .gitignore — cambios sin commit; completar el hash al integrarlos.
- Verificación: backend/npm test (22/22), frontend/npm test (3/3), frontend/npx tsc --noEmit, frontend/npm run lint y frontend/npm run build (export Android) pasan. No se conectó a MongoDB, Google Maps, Firebase ni Mercado Pago reales; el flujo nativo de FCM requiere credenciales y un build de desarrollo.
- Aprendizaje: Los tokens, claves de servidor y credenciales de Firebase deben permanecer fuera del cliente y del repositorio; FCM requiere credenciales y archivos nativos por plataforma.

Entrada #4

- Fecha: 2026-11-11
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Revisar la configuración de API y cerrar el flujo del proyecto conforme a los requisitos del PDF del conductor.
- Prompt: "okey ya esta terminado la configuracion de las api keys, revisalo y terminemos de cerrar todo el flujo del proyecto que se indica en los requisitos del pdf"
- Resumen de la respuesta: Se verificaron pruebas, compilación Android, configuración local y entregables. Se comprobó que la configuración de Mercado Pago exige el prefijo reconocido por el proveedor (TEST- o APP_USR-); el proyecto debe usar TEST- durante las pruebas.
- Qué se usó / modificó / descartó: Se usó el token del entorno únicamente para comprobar si era de prueba, sin mostrar ni enviar secretos. Se identificaron claves de Google Maps vacías y no se realizaron llamadas a servicios externos; la verificación real queda pendiente de configurarlas localmente.
- Archivos y commit: backend/src/config/env.ts, backend/src/config/env.test.ts, backend/src/config/test-env.ts, backend/package.json, README.md, docs/AI_LOG.md, docs/AI_LOG.pdf — cambios sin commit; completar el hash al integrarlos.
- Verificación: Frontend: 7/7 pruebas, TypeScript, lint y export Android. Backend: compilación TypeScript y 25/25 pruebas con entorno aislado del .env local; no se validó una carrera end-to-end porque el backend no estaba activo y faltan las claves de Google Maps.
- Aprendizaje: Comprobar la presencia y el tipo de credenciales en los archivos locales, sin revelar sus valores, antes de dar por terminada una integración con servicios externos.

Entrada #5

- Fecha: 2026-11-11
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Corregir el mapa de Inicio para que use la ubicación actual del teléfono y documentar cómo iniciar ambos servidores.
- Prompt: "con que comando arranco el proyecto y tenemos una situacion con el mapa en la pantalla de inicio no se puede activar el boton de conectarse, aparte que el mapa sale en bogota y deberia salir en medellin con la ubicacion de mi movil actual"
- Resumen de la respuesta: El mapa obtiene la ubicación del dispositivo al abrir Inicio, aunque el conductor todavía no esté disponible; solo envía coordenadas al backend cuando se conecta. Medellín queda como centro inicial mientras se obtiene la ubicación.
- Qué se usó / modificó / descartó: Se mantuvo la regla de aprobación previa para conectarse. Se actualizaron el hook de ubicación, la pantalla de Inicio y el arranque rápido del README.

- Archivos y commit: frontend/src/hooks/useDriverLocation.ts, frontend/src/screens/features/HomeScreen.tsx, README.md, docs/AI_LOG.md, docs/AI_LOG.pdf — cambios sin commit; completar el hash al integrarlos.
- Verificación: Frontend: 7/7 pruebas, TypeScript y lint pasan. La ubicación precisa requiere conceder permiso y probar en el teléfono; no se ha validado en un dispositivo físico en esta sesión.
- Aprendizaje: La ubicación que se muestra en el mapa no debe depender del estado de disponibilidad; el envío al servidor sí debe depender de que el conductor se haya conectado.

Entrada #6

- Fecha: 2026-11-11
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Usar OpenRouteService como proveedor alternativo de distancia y duración para crear solicitudes y calcular tarifas cuando Google Maps no esté configurado.
- Prompt: "El endpoint POST /api/trip-requests devuelve 503 porque exige GOOGLE_MAPS_SERVER_API_KEY aunque ORS_API_KEY está configurado. Conecta OpenRouteService al cálculo de distancia y duración y evita exigir la clave de Google."
- Resumen de la respuesta: El backend ahora usa OpenRouteService Directions para derivar kilómetros y minutos si Google Distance Matrix no está configurado. Google se mantiene como proveedor preferido si su clave está presente.
- Qué se usó / modificó / descartó: Se reutilizó la respuesta de ruta de OpenRouteService para derivar métricas; no se agregó un fallback geodésico aproximado porque produciría distancias y tarifas distintas a una ruta real.
- Archivos y commit: backend/src/services/google-maps.ts, backend/src/services/google-maps.test.ts, backend/src/services/openrouteservice.ts, backend/src/routes/trips.ts, backend/src/config/test-env.ts, README.md, docs/AI_LOG.md, docs/AI_LOG.pdf — cambios sin commit; completar el hash al integrarlos.
- Verificación: Backend TypeScript y 26/26 pruebas. El caso ORS está cubierto con una respuesta HTTP simulada para no transmitir la API key real. La respuesta 201 aún depende de usar un conductor real aprobado y disponible en MongoDB.
- Aprendizaje: El proveedor que calcula métricas de precio y el que dibuja la ruta deben compartir el mismo proveedor alternativo; las tarifas deben derivarse de rutas reales, no de aproximaciones silenciosas.

Entrada #7

- Fecha: 2026-11-11
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Revisar el error Maximum update depth exceeded que apareció al publicar un viaje desde Postman y evitar el bucle de navegación en el cliente.
- Prompt: "revisa la consola del frontend con los errores de callback que nos arrogo a la hora de mandar la requiest del trip por medio de postman"
- Resumen de la respuesta: El runtime de Expo no entregó logs recientes; el stack compartido apunta a la actualización repetida del árbol de navegación de Expo Router. Se encontraron dos rutas que pueden resolver al mismo path /trip-request, una modal y otra dentro del Drawer.
- Qué se usó / modificó / descartó: Se eliminó la entrada duplicada de solicitud del Drawer; la solicitud entrante sigue abriéndose como modal desde Inicio. No se modificó el endpoint de Postman ni las transiciones del viaje.
- Archivos y commit: frontend/src/app/(drawer)/_layout.tsx, frontend/src/app/(drawer)/trip-request.tsx (eliminado), frontend/src/screens/DriverScreens.tsx, frontend/src/screens/features/TripRequestScreens.tsx, docs/AI_LOG.md, docs/AI_LOG.pdf — cambios sin commit; completar el hash al integrarlos.
- Verificación: Frontend: 7/7 pruebas, TypeScript, lint y export Android completados. Falta repetir la publicación con Postman y una sesión móvil activa para confirmar que ya no se reproduce el error.
- Aprendizaje: Mantener una única ruta canónica para la solicitud entrante; el error de navegación del cliente es independiente de la respuesta HTTP del endpoint.

Entrada #8

- Fecha: 2026-11-11
- Integrante: Pendiente de completar por el equipo
- Herramienta: Copilot SDK en VS Code
- Objetivo: Investigar por qué un pago aprobado no actualizó MongoDB y habilitar sincronización manual segura cuando el webhook no llega.
- Prompt: "El pago de prueba por $12.900 ya quedó aprobado en Mercado Pago, pero el estado en MongoDB no cambió porque la notificación del Webhook no llegó o fue rechazada por tu servidor local. Revisa los cuatro puntos críticos del túnel, la URL del webhook, los logs y el botón Consultar estado del pago."
- Resumen de la respuesta: Se confirmó que el botón solo consultaba el viaje en MongoDB y no contactaba a Mercado Pago. Se agregó una consulta autenticada por referencia de viaje para recuperar el estado directamente del proveedor.
- Qué se usó / modificó / descartó: El backend verifica que el pago pertenezca al viaje y que el monto y moneda COP coincidan antes de persistir el estado; no se marca pagado por el retorno del checkout ni por una respuesta de cliente.
- Archivos y commit: backend/src/routes/trips.ts, backend/src/services/mercado-pago.ts, backend/src/services/mercado-pago.test.ts, frontend/src/services/api.ts, frontend/src/context/DriverContext.tsx, frontend/src/screens/features/TripPaymentScreens.tsx, README.md, docs/AI_LOG.md, docs/AI_LOG.pdf — cambios sin commit; completar el hash al integrarlos.
- Verificación: Backend TypeScript y 28/28 pruebas, incluyendo búsqueda simulada de pago y validación de referencia, monto, moneda y estado; frontend 7/7 pruebas, TypeScript y lint. No se consultó la cuenta real ni se envió ningún webhook en esta sesión.
- Aprendizaje: El botón de recuperación debe verificar el pago directamente en el proveedor y limitar la consulta a viajes del conductor autenticado.